CineComic-MML

Phase 1: Video Ingestion and Validation - complete code

This document contains every file needed for Phase 0 (the foundation Phase 1 depends on) and Phase 1, laid out in the folder structure from the guide. All code was run and tested: **15 unit tests pass**, and the command-line run on a sample clip passes the Phase 1 gate.

What Phase 1 does

• Accepts a local video or a URL (yt-dlp), computes SHA-256 and a video_id (clip_001 or a hash prefix).

• Reads duration, fps, resolution, codec, rotation, VFR and audio stream with ffprobe.

• Decode-tests the file; any ffmpeg error line counts as corruption.

• Re-encodes to constant frame rate / upright / mp4 only when needed (VFR, rotation, non-mp4, over max duration), so that time = frame_index / fps holds for the whole project.

• Extracts 16 kHz mono WAV (skipped when the video has no audio) and saves metadata JSON.

• Evaluates and applies the gate; results are cached by config hash.

Folder structure (Phase 1 files marked)

```
CineComic_MML/
|-- main.py                          entry point
|-- env_check.py                     Phase 0 environment report
|-- setup_project.py                 creates the whole folder skeleton
|-- requirements.txt   README.md
|-- configs/
|   `-- phase1.yaml
|-- src/
|   |-- common/    utils.py  base_stage.py
|   `-- ingestion/ metadata.py  video_loader.py  video_ingestion.py
|-- tests/         conftest.py  test_phase1.py
|-- data/  raw/ metadata/ audio/ ...  (created by setup_project.py)
|-- results/  logs/  notebooks/  models/
```

Quick start

```
python -m venv .venv  &&  .venv\Scripts\activate      (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt           (FFmpeg + ffprobe must also be on PATH)
python setup_project.py
python env_check.py
python -m pytest -q tests
python main.py --config configs/phase1.yaml --video path\to\clip.mp4 --video-id clip_001
```

Artifacts written for clip_001

File	Content
data/raw/clip_001.mp4	Normalised CFR, upright H.264/AAC mp4
data/metadata/clip_001.json	video_id, fps, duration_s, width, height, n_frames, has_audio, audio_path, vfr, sha256 ...
data/metadata/clip_001.metrics.json	Gate metrics, pass/fail, config hash, runtime
data/audio/clip_001.wav	16 kHz mono PCM (absent if no audio)

Phase 1 gate (from the guide)

Metric	Gate	Config key
Frame count: OpenCV vs ffprobe	<= 1%	gates.frame_count_rel_diff
|audio duration - video duration|	<= 0.1 s	gates.audio_video_diff_s
ffmpeg null-decode error lines	0	gates.decode_errors
Random frame check (20 seeks)	no black/unreadable frames (10% tolerance)	ingestion.max_bad_frame_fraction

Note: the PowerShell block in the guide contains two typos (`data\facesmkdir data\tracks` and `resultsmkdir logs` were joined on one line). setup_project.py avoids this and also creates data/speaker, which Section 12 lists.

1. setup_project.py

Creates the full folder skeleton (replaces the PowerShell mkdir / New-Item commands).

```
"""Create the CineComic_MML folder skeleton (cross-platform replacement for the
PowerShell mkdir / New-Item commands in the guide).

Usage:  python setup_project.py
"""
from pathlib import Path

DATA = ["raw", "metadata", "audio", "shots", "frames", "subtitles", "diarization",
        "faces", "tracks", "embeddings", "alignment", "speaker", "keyframes",
        "emotions", "layout", "outputs"]
SRC = ["common", "ingestion", "audio", "video", "vision", "speaker", "alignment",
       "keyframes", "emotion", "layout", "comic", "evaluation"]
TOP = ["configs", "tests", "results", "logs", "notebooks", "models"]


def main():
    root = Path(__file__).resolve().parent
    dirs = [root / "data" / d for d in DATA]
    dirs += [root / "src" / s for s in SRC]
    dirs += [root / t for t in TOP]
    for d in dirs:
        d.mkdir(parents=True, exist_ok=True)
    (root / "src" / "__init__.py").touch()
    for s in SRC:
        (root / "src" / s / "__init__.py").touch()
    print(f"Created {len(dirs)} folders under {root}")


if __name__ == "__main__":
    main()
```

2. requirements.txt

Python dependencies for Phase 0 and 1.

```
# Phase 0/1 dependencies. After your first successful install run
# `pip freeze > requirements.lock.txt` and keep that file for exact reproducibility.
# System tools (not pip): FFmpeg + ffprobe must be on PATH.
PyYAML>=6.0,<7
numpy>=1.26,<3
opencv-python>=4.8,<5
yt-dlp>=2024.1.1
pytest>=8.0,<10
```

3. README.md

Setup, run and gate instructions.

```
# CineComic-MML

Multimodal video-to-comic system, built one stage at a time. This repository currently
contains **Phase 0 (foundation) and Phase 1 (video ingestion and validation)**.

## Setup
1. Install Python 3.10/3.11+ and FFmpeg (`ffmpeg -version` and `ffprobe -version` must work).
2. `python -m venv .venv` and activate it, then `pip install -r requirements.txt`
3. `python setup_project.py` to create the folder skeleton
4. `python env_check.py` -> writes `results/environment_report.json`

## Run Phase 1
```
python main.py --config configs/phase1.yaml --video path/to/clip.mp4 --video-id clip_001
```
Outputs:
- `data/raw/clip_001.mp4`          normalised constant-frame-rate, upright mp4
- `data/metadata/clip_001.json`   metadata (time convention: float seconds, frame = round(t*fps))
- `data/metadata/clip_001.metrics.json`  gate metrics and pass/fail
- `data/audio/clip_001.wav`       16 kHz mono WAV (skipped if the video has no audio)

Exit code: 0 gate passed, 1 gate failed, 2 input rejected.

## Tests
`python -m pytest -q tests`

## Phase 1 gate
frame-count agreement <= 1%, |audio - video length| <= 0.1 s, decode errors = 0,
and the random-frame check finds no (or very few) black/unreadable frames.
Thresholds live in `configs/phase1.yaml`.
```

4. configs/phase1.yaml

All Phase 1 thresholds and settings (nothing hard-coded in src/).

```yaml
# Phase 1 - Video ingestion and validation.
# Every threshold lives here; nothing is hard-coded in src/.
data_dir: data
results_dir: results
log_dir: logs
seed: 42

ingestion:
  max_duration_s: 300        # development cap; 0 = no limit
  min_duration_s: 1.0        # reject clips shorter than this
  audio_sample_rate: 16000   # Whisper / pyannote expect 16 kHz mono
  force_cfr: true            # re-encode variable frame rate to constant frame rate
  vfr_tolerance: 0.02        # relative gap between r_frame_rate and avg_frame_rate
  random_frame_checks: 20    # number of random seek-and-read checks
  black_frame_mean: 8.0      # mean pixel value (0-255) below which a frame counts as black
  max_bad_frame_fraction: 0.1  # tolerated share of black/unreadable sampled frames
  x264_crf: 18               # quality used only when a re-encode is needed
  x264_preset: veryfast

gates:                       # Phase 1 pass/fail thresholds (Section 5, Phase 1)
  frame_count_rel_diff: 0.01 # OpenCV vs ffprobe frame count, relative
  audio_video_diff_s: 0.1    # |audio duration - video duration|
  decode_errors: 0           # lines printed by the ffmpeg null decode
```

5. src/common/utils.py

Phase 0 shared utilities: load_config (rejects unknown keys), set_seed, get_logger, artifact_path.

```
"""Phase 0 shared utilities: config loader, seeding, logging, artifact paths."""
from __future__ import annotations

import hashlib
import json
import logging
import random
from pathlib import Path

import yaml


class ConfigError(ValueError):
    """Raised when a config file contains unknown or invalid keys."""


# Allowed keys. A value of None means "any value"; a dict means "nested section".
# Later phases extend this schema when they add their own config sections.
CONFIG_SCHEMA = {
    "data_dir": None,
    "results_dir": None,
    "log_dir": None,
    "seed": None,
    "ingestion": {
        "max_duration_s": None,
        "min_duration_s": None,
        "audio_sample_rate": None,
        "force_cfr": None,
        "vfr_tolerance": None,
        "random_frame_checks": None,
        "black_frame_mean": None,
        "max_bad_frame_fraction": None,
        "x264_crf": None,
        "x264_preset": None,
    },
    "gates": {
        "frame_count_rel_diff": None,
        "audio_video_diff_s": None,
        "decode_errors": None,
    },
}


def _validate(cfg: dict, schema: dict, prefix: str = "") -> None:
    for key, value in cfg.items():
        path = f"{prefix}{key}"
        if key not in schema:
            raise ConfigError(f"Unknown config key: '{path}'")
        sub = schema[key]
        if isinstance(sub, dict):
            if not isinstance(value, dict):
                raise ConfigError(f"Config key '{path}' must be a mapping")
            _validate(value, sub, prefix=f"{path}.")


def load_config(path: str | Path, schema: dict | None = None) -> dict:
    """Load a YAML config and reject unknown keys."""
    with open(path, "r", encoding="utf-8") as fh:
        cfg = yaml.safe_load(fh) or {}
    if not isinstance(cfg, dict):
        raise ConfigError("Top level of the config must be a mapping")
    _validate(cfg, schema or CONFIG_SCHEMA)
    return cfg


def config_hash(cfg: dict) -> str:
    """Short stable hash of a config, used for caching and in metrics files."""
    blob = json.dumps(cfg, sort_keys=True, default=str).encode("utf-8")
    return hashlib.sha1(blob).hexdigest()[:8]


def set_seed(seed: int) -> None:
    """Seed Python, NumPy and (if installed) PyTorch."""
    random.seed(seed)
    try:
        import numpy as np
        np.random.seed(seed)
    except ImportError:
        pass
    try:
        import torch
        torch.manual_seed(seed)
```

```
    except ImportError:
        pass


def get_logger(video_id: str, log_dir: str | Path = "logs") -> logging.Logger:
    """Logger that writes to the console and to logs/{video_id}.log."""
    logger = logging.getLogger(f"cinecomic.{video_id}")
    if logger.handlers:                     # already configured in this process
        return logger
    logger.setLevel(logging.INFO)
    logger.propagate = False
    fmt = logging.Formatter("%(asctime)s | %(levelname)-7s | %(name)s | %(message)s")
    Path(log_dir).mkdir(parents=True, exist_ok=True)
    file_handler = logging.FileHandler(Path(log_dir) / f"{video_id}.log", encoding="utf-8")
    file_handler.setFormatter(fmt)
    console = logging.StreamHandler()
    console.setFormatter(fmt)
    logger.addHandler(file_handler)
    logger.addHandler(console)
    return logger


def close_logger(logger: logging.Logger) -> None:
    """Release file handles (needed on Windows and in tests)."""
    for handler in list(logger.handlers):
        handler.close()
        logger.removeHandler(handler)


def artifact_path(cfg: dict, kind: str, video_id: str, ext: str = "json") -> Path:
    """Return data/{kind}/{video_id}.{ext} and create the parent folder.

    Example: artifact_path(cfg, "audio", "clip_001", "wav") -> data/audio/clip_001.wav
    """
    path = Path(cfg["data_dir"]) / kind / f"{video_id}.{ext}"
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

6. src/common/base_stage.py

BaseStage contract: run, evaluate, gate, metrics file, config-hash cache.

```python
"""BaseStage: the contract every phase follows (Section 12.1 of the guide).

input artifacts in -> run() -> one saved artifact -> evaluate() -> gate() ->
metrics JSON saved next to the artifact. Results are cached by config hash.
"""
from __future__ import annotations

import json
import platform
import time
from pathlib import Path

from src.common.utils import config_hash


class BaseStage:
    name = "base"
    requires: list[str] = []            # names of earlier stages

    def __init__(self, cfg: dict, video_id: str, logger=None):
        self.cfg = cfg
        self.video_id = video_id
        self.log = logger

    # ---- to be provided by subclasses ----------------------------------
    @property
    def artifact(self) -> Path:
        """Path of the main artifact this stage writes."""
        raise NotImplementedError

    def run(self) -> None:
        raise NotImplementedError

    def evaluate(self, gt=None) -> dict:
        raise NotImplementedError

    def gate(self, metrics: dict) -> tuple[bool, str]:
        raise NotImplementedError

    # ---- shared behaviour ----------------------------------------------
    @property
    def metrics_path(self) -> Path:
        return self.artifact.parent / f"{self.video_id}.metrics.json"

    def cache_key(self) -> str:
        return config_hash(self.cfg)

    def is_cached(self) -> bool:
        """True if artifact + metrics exist, the config is unchanged and the gate passed."""
        if not (self.artifact.exists() and self.metrics_path.exists()):
            return False
        try:
            saved = json.loads(self.metrics_path.read_text(encoding="utf-8"))
        except json.JSONDecodeError:
            return False
        return saved.get("cfg") == self.cache_key() and saved.get("passed") is True

    def execute(self, gt=None, force: bool = False) -> bool:
        if not force and self.is_cached():
            if self.log:
                self.log.info("[%s] cached (config unchanged), skipping", self.name)
            return True
        t0 = time.time()
        self.run()
        metrics = self.evaluate(gt)
        ok, reason = self.gate(metrics)
        metrics.update(
            stage=self.name,
            video_id=self.video_id,
            runtime_s=round(time.time() - t0, 3),
            passed=ok,
            reason=reason,
            cfg=self.cache_key(),
            python=platform.python_version(),
        )
        self.metrics_path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
        if self.log:
            self.log.info("[%s] gate %s: %s", self.name, "PASSED" if ok else "FAILED", reason)
        return ok
```

7. src/ingestion/metadata.py

ffprobe and OpenCV helpers: describe, decode test, frame count, random-frame check.

```
"""ffprobe / OpenCV helpers: describe a video and run the Phase 1 checks."""
from __future__ import annotations

import json
import random
import subprocess
import wave
from pathlib import Path


class IngestionError(RuntimeError):
    """Raised when a video cannot be accepted (missing, corrupt, too short...)."""


def _run(cmd: list[str]) -> subprocess.CompletedProcess:
    return subprocess.run(cmd, capture_output=True, text=True, encoding="utf-8",
                          errors="replace")


def _fraction(text: str | None) -> float:
    """'30000/1001' -> 29.97...; '0/0' or None -> 0.0"""
    if not text:
        return 0.0
    if "/" in text:
        num, den = text.split("/", 1)
        den_f = float(den)
        return float(num) / den_f if den_f else 0.0
    return float(text)


def probe(path: str | Path) -> dict:
    """Run ffprobe and return the parsed JSON (format + streams)."""
    proc = _run(["ffprobe", "-v", "error", "-print_format", "json",
                 "-show_format", "-show_streams", str(path)])
    if proc.returncode != 0:
        raise IngestionError(f"ffprobe could not read '{path}' "
                             f"(file corrupt or truncated?): {proc.stderr.strip()[:200]}")
    try:
        return json.loads(proc.stdout)
    except json.JSONDecodeError as exc:
        raise IngestionError(f"ffprobe returned invalid JSON for '{path}'") from exc


def _rotation(vstream: dict) -> int:
    """Rotation in degrees from the legacy 'rotate' tag or the display matrix."""
    tag = vstream.get("tags", {}).get("rotate")
    if tag is not None:
        return int(float(tag)) % 360
    for side in vstream.get("side_data_list", []):
        if "rotation" in side:
            return int(round(float(side["rotation"]))) % 360
    return 0


def describe(path: str | Path, vfr_tolerance: float = 0.02) -> dict:
    """Summarise a video file. Width/height are the *displayed* size (after rotation)."""
    info = probe(path)
    vstreams = [s for s in info.get("streams", []) if s.get("codec_type") == "video"]
    astreams = [s for s in info.get("streams", []) if s.get("codec_type") == "audio"]
    if not vstreams:
        raise IngestionError(f"'{path}' has no video stream")
    v = vstreams[0]

    rate = _fraction(v.get("r_frame_rate"))
    avg = _fraction(v.get("avg_frame_rate"))
    fps = avg or rate
    if fps <= 0:
        raise IngestionError("Could not determine the frame rate")
    vfr = bool(rate and avg and abs(rate - avg) / rate > vfr_tolerance)

    duration = float(info.get("format", {}).get("duration") or v.get("duration") or 0)
    if duration <= 0:
        raise IngestionError("Could not determine the duration")

    width, height = int(v.get("width", 0)), int(v.get("height", 0))
    rotation = _rotation(v)
    if rotation in (90, 270):
        width, height = height, width
    if width <= 0 or height <= 0:
        raise IngestionError("Invalid video resolution")

    nb = v.get("nb_frames")
```

```
    n_frames = int(nb) if nb and str(nb).isdigit() else None
    return {
        "duration_s": duration,
        "fps": fps,
        "fps_fraction": v.get("avg_frame_rate") if avg else v.get("r_frame_rate"),
        "width": width,
        "height": height,
        "rotation": rotation,
        "video_codec": v.get("codec_name"),
        "n_frames_ffprobe": n_frames,
        "n_frames": n_frames if n_frames is not None else int(round(duration * fps)),
        "n_frames_estimated": n_frames is None,
        "vfr": vfr,
        "has_audio": bool(astreams),
        "audio_codec": astreams[0].get("codec_name") if astreams else None,
    }


def decode_errors(path: str | Path) -> list[str]:
    """Decode the whole file to nowhere; every stderr line counts as an error."""
    proc = _run(["ffmpeg", "-nostdin", "-v", "error", "-i", str(path), "-f", "null", "-"])
    return [line for line in proc.stderr.splitlines() if line.strip()]


def opencv_frame_count(path: str | Path) -> int:
    import cv2
    cap = cv2.VideoCapture(str(path))
    try:
        if not cap.isOpened():
            raise IngestionError(f"OpenCV cannot open '{path}'")
        return int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    finally:
        cap.release()


def random_frame_check(path: str | Path, duration_s: float, n: int, seed: int,
                       black_mean: float) -> dict:
    """Seek to n random timestamps; a frame is bad if unreadable or (almost) black."""
    import cv2
    rng = random.Random(seed)
    times = sorted(rng.uniform(0.0, max(duration_s - 0.2, 0.0)) for _ in range(n))
    bad = []
    cap = cv2.VideoCapture(str(path))
    try:
        for t in times:
            cap.set(cv2.CAP_PROP_POS_MSEC, t * 1000.0)
            ok, frame = cap.read()
            if not ok or frame is None or float(frame.mean()) < black_mean:
                bad.append(round(t, 3))
    finally:
        cap.release()
    return {"checked": n, "bad": len(bad), "bad_times_s": bad}


def wav_duration(path: str | Path) -> float:
    with wave.open(str(path), "rb") as wf:
        return wf.getnframes() / float(wf.getframerate())


def wav_format(path: str | Path) -> tuple[int, int]:
    """Return (sample_rate, channels) of a WAV file."""
    with wave.open(str(path), "rb") as wf:
        return wf.getframerate(), wf.getnchannels()
```

8. src/ingestion/video_loader.py

Acquire file/URL, SHA-256 video_id, normalise to CFR/upright mp4, extract 16 kHz mono WAV.

```
"""Acquire a video (file or URL), hash it, normalise it and extract 16 kHz audio."""
from __future__ import annotations

import hashlib
import shutil
import subprocess
import tempfile
from pathlib import Path

from src.ingestion.metadata import IngestionError


def is_url(text: str) -> bool:
    return text.lower().startswith(("http://", "https://"))


def require_tools() -> None:
    missing = [t for t in ("ffmpeg", "ffprobe") if shutil.which(t) is None]
    if missing:
        raise IngestionError(f"Required tool(s) not found on PATH: {', '.join(missing)}")


def sha256_file(path: str | Path, chunk: int = 1 << 20) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as fh:
        while block := fh.read(chunk):
            h.update(block)
    return h.hexdigest()


def make_video_id(sha256: str, requested: str | None = None) -> str:
    """Use the caller's id (e.g. 'clip_001') or the first 12 hex chars of the SHA-256."""
    if requested:
        if not requested.replace("_", "").replace("-", "").isalnum():
            raise IngestionError("video_id may only contain letters, digits, '_' and '-'")
        return requested
    return sha256[:12]


def download_url(url: str, dest_dir: str | Path) -> Path:
    """Download a URL with yt-dlp into dest_dir and return the file path."""
    try:
        import yt_dlp
    except ImportError as exc:
        raise IngestionError("yt-dlp is not installed (pip install yt-dlp)") from exc
    opts = {"outtmpl": str(Path(dest_dir) / "download.%(ext)s"), "quiet": True,
            "merge_output_format": "mp4", "noplaylist": True}
    with yt_dlp.YoutubeDL(opts) as ydl:
        ydl.download([url])
    files = sorted(Path(dest_dir).glob("download.*"))
    if not files:
        raise IngestionError(f"Download produced no file for {url}")
    return files[0]


def acquire_source(source: str, work_dir: str | Path | None = None) -> Path:
    """Return a local path for a file path or URL."""
    if is_url(source):
        work = Path(work_dir) if work_dir else Path(tempfile.mkdtemp(prefix="cinecomic_dl_"))
        work.mkdir(parents=True, exist_ok=True)
        return download_url(source, work)
    path = Path(source)
    if not path.is_file():
        raise IngestionError(f"Input file not found: {path}")
    if path.stat().st_size == 0:
        raise IngestionError(f"Input file is empty: {path}")
    return path


def needs_normalising(src: Path, info: dict, icfg: dict) -> tuple[bool, list[str]]:
    """Decide whether the source must be re-encoded; also return the reasons."""
    reasons = []
    if src.suffix.lower() != ".mp4":
        reasons.append("container is not mp4")
    if info["vfr"] and icfg["force_cfr"]:
        reasons.append("variable frame rate")
    if info["rotation"] != 0:
        reasons.append(f"rotation tag {info['rotation']}")
    max_d = icfg["max_duration_s"]
    if max_d and info["duration_s"] > max_d:
        reasons.append(f"longer than max_duration_s={max_d}")
    return bool(reasons), reasons
```

```
 83
 84
 85  def normalise_video(src: Path, dst: Path, info: dict, icfg: dict) -> None:
 86      """Re-encode to constant frame rate, upright orientation, H.264/AAC mp4.
 87
 88      FFmpeg applies the rotation tag automatically while transcoding and removes it,
 89      so the output frames are upright with no rotation metadata left.
 90      """
 91      cmd = ["ffmpeg", "-y", "-nostdin", "-v", "error", "-i", str(src)]
 92      max_d = icfg["max_duration_s"]
 93      if max_d and info["duration_s"] > max_d:
 94          cmd += ["-t", str(max_d)]
 95      cmd += ["-map", "0:v:0", "-map", "0:a:0?"]
 96      if icfg["force_cfr"]:
 97          cmd += ["-vf", f"fps={info['fps_fraction']}"]
 98      cmd += ["-c:v", "libx264", "-preset", str(icfg["x264_preset"]),
 99              "-crf", str(icfg["x264_crf"]), "-pix_fmt", "yuv420p",
100              "-c:a", "aac", "-b:a", "192k", "-movflags", "+faststart", str(dst)]
101      proc = subprocess.run(cmd, capture_output=True, text=True, errors="replace")
102      if proc.returncode != 0:
103          raise IngestionError(f"ffmpeg re-encode failed: {proc.stderr.strip()[:300]}")
104
105
106  def copy_video(src: Path, dst: Path) -> None:
107      shutil.copy2(src, dst)
108
109
110  def extract_audio(video: Path, wav: Path, sample_rate: int) -> None:
111      """16 kHz mono 16-bit PCM WAV (what Whisper and pyannote expect)."""
112      cmd = ["ffmpeg", "-y", "-nostdin", "-v", "error", "-i", str(video), "-vn",
113             "-ac", "1", "-ar", str(sample_rate), "-c:a", "pcm_s16le", str(wav)]
114      proc = subprocess.run(cmd, capture_output=True, text=True, errors="replace")
115      if proc.returncode != 0:
116          raise IngestionError(f"Audio extraction failed: {proc.stderr.strip()[:300]}")
```

9. src/ingestion/video_ingestion.py

The Phase 1 stage: writes raw mp4, metadata JSON, WAV; evaluates and applies the gate.

```
"""Phase 1 stage: ingest a video and produce raw mp4, metadata JSON and 16 kHz WAV."""
from __future__ import annotations

import json
import tempfile
from datetime import datetime
from pathlib import Path

from src.common.base_stage import BaseStage
from src.common.utils import artifact_path
from src.ingestion import metadata as md
from src.ingestion import video_loader as vl


class VideoIngestionStage(BaseStage):
    name = "ingestion"
    requires: list[str] = []

    def __init__(self, cfg, video_id, source_path: Path, sha256: str, source_label: str,
                 logger=None):
        super().__init__(cfg, video_id, logger)
        self.source_path = Path(source_path)
        self.sha256 = sha256
        self.source_label = source_label
        self.raw_path = artifact_path(cfg, "raw", video_id, "mp4")
        self.wav_path = artifact_path(cfg, "audio", video_id, "wav")
        self._artifact = artifact_path(cfg, "metadata", video_id, "json")

    @property
    def artifact(self) -> Path:
        return self._artifact

    # ---- processing ------------------------------------------------------
    def run(self) -> None:
        icfg = self.cfg["ingestion"]
        vl.require_tools()

        src_info = md.describe(self.source_path, icfg["vfr_tolerance"])
        if src_info["duration_s"] < icfg["min_duration_s"]:
            raise md.IngestionError(
                f"Video too short ({src_info['duration_s']:.2f}s < {icfg['min_duration_s']}s)")
        src_errors = md.decode_errors(self.source_path)

        redo, reasons = vl.needs_normalising(self.source_path, src_info, icfg)
        if redo:
            self._info("re-encoding source: %s", ", ".join(reasons))
            vl.normalise_video(self.source_path, self.raw_path, src_info, icfg)
        else:
            vl.copy_video(self.source_path, self.raw_path)

        raw = md.describe(self.raw_path, icfg["vfr_tolerance"])   # describe the final file

        audio_path, audio_dur = None, None
        if raw["has_audio"]:
            vl.extract_audio(self.raw_path, self.wav_path, icfg["audio_sample_rate"])
            audio_path, audio_dur = self.wav_path.as_posix(), md.wav_duration(self.wav_path)
        else:
            self._info("no audio stream: skipping audio extraction")
            if self.wav_path.exists():
                self.wav_path.unlink()          # drop a stale WAV from an earlier run

        meta = {
            "video_id": self.video_id,
            "source": self.source_label,
            "sha256": self.sha256,
            "raw_path": self.raw_path.as_posix(),
            "duration_s": round(raw["duration_s"], 3),
            "fps": round(raw["fps"], 5),
            "fps_fraction": raw["fps_fraction"],
            "width": raw["width"],
            "height": raw["height"],
            "n_frames": raw["n_frames"],
            "video_codec": raw["video_codec"],
            "has_audio": raw["has_audio"],
            "audio_path": audio_path,
            "audio_duration_s": round(audio_dur, 3) if audio_dur is not None else None,
            "audio_sample_rate": icfg["audio_sample_rate"] if audio_path else None,
            "vfr": raw["vfr"],
            "normalised": redo,
            "normalise_reasons": reasons,
            "source_vfr": src_info["vfr"],
            "source_rotation": src_info["rotation"],
```

```
            "source_decode_errors": len(src_errors),
            "time_convention": "seconds (float) from start of CFR video; frame = round(t*fps)",
            "created": datetime.now().isoformat(timespec="seconds"),
        }
        self.artifact.write_text(json.dumps(meta, indent=2), encoding="utf-8")

    # ---- evaluation ----------------------------------------------------------
    def evaluate(self, gt=None) -> dict:
        icfg = self.cfg["ingestion"]
        meta = json.loads(self.artifact.read_text(encoding="utf-8"))

        cv_frames = md.opencv_frame_count(self.raw_path)
        ref = max(meta["n_frames"], 1)
        frame_diff = abs(cv_frames - meta["n_frames"]) / ref

        av_diff = None
        if meta["has_audio"]:
            av_diff = abs(meta["audio_duration_s"] - meta["duration_s"])

        errs = max(meta["source_decode_errors"], len(md.decode_errors(self.raw_path)))
        frames = md.random_frame_check(self.raw_path, meta["duration_s"],
                                       icfg["random_frame_checks"], self.cfg["seed"],
                                       icfg["black_frame_mean"])
        return {
            "opencv_frames": cv_frames,
            "ffprobe_frames": meta["n_frames"],
            "frame_count_rel_diff": round(frame_diff, 5),
            "audio_video_diff_s": None if av_diff is None else round(av_diff, 4),
            "decode_errors": errs,
            "random_frame_check": frames,
            "has_audio": meta["has_audio"],
        }

    def gate(self, metrics: dict) -> tuple[bool, str]:
        g, icfg = self.cfg["gates"], self.cfg["ingestion"]
        fails = []
        if metrics["frame_count_rel_diff"] > g["frame_count_rel_diff"]:
            fails.append(f"frame count diff {metrics['frame_count_rel_diff']:.4f} "
                         f"> {g['frame_count_rel_diff']}")
        diff = metrics["audio_video_diff_s"]
        if diff is not None and diff > g["audio_video_diff_s"]:
            fails.append(f"audio/video length diff {diff}s > {g['audio_video_diff_s']}s")
        if metrics["decode_errors"] > g["decode_errors"]:
            fails.append(f"{metrics['decode_errors']} decode error line(s)")
        chk = metrics["random_frame_check"]
        if chk["bad"] > icfg["max_bad_frame_fraction"] * chk["checked"]:
            fails.append(f"{chk['bad']}/{chk['checked']} random frames black or unreadable")
        return (not fails, "all checks passed" if not fails else "; ".join(fails))

    def _info(self, msg, *args):
        if self.log:
            self.log.info(msg, *args)


def ingest(cfg: dict, source: str, video_id: str | None = None, logger_factory=None,
           force: bool = False) -> tuple[bool, str]:
    """Convenience entry point used by main.py. Returns (passed, video_id)."""
    from src.common.utils import get_logger
    vl.require_tools()
    with tempfile.TemporaryDirectory(prefix="cinecomic_dl_") as tmp:
        local = vl.acquire_source(source, tmp)
        sha = vl.sha256_file(local)
        vid = vl.make_video_id(sha, video_id)
        log = (logger_factory or get_logger)(vid, cfg["log_dir"])
        log.info("ingesting %s as %s", source, vid)
        stage = VideoIngestionStage(cfg, vid, local, sha, source, log)
        ok = stage.execute(force=force)
    return ok, vid
```

10. main.py

Command-line entry point.

```
"""CineComic-MML entry point.

Phase 1 usage:
    python main.py --config configs/phase1.yaml --video path/to/clip.mp4 --video-id clip_001
    python main.py --config configs/phase1.yaml --video https://example.com/video
Later phases will register their stages in STAGES below.
"""
from __future__ import annotations

import argparse
import sys

from src.common.utils import load_config, set_seed
from src.ingestion.metadata import IngestionError
from src.ingestion.video_ingestion import ingest

STAGES = ["ingestion"]          # extended phase by phase


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="CineComic-MML pipeline")
    ap.add_argument("--config", required=True, help="YAML config, e.g. configs/phase1.yaml")
    ap.add_argument("--video", required=True, help="video file path or URL")
    ap.add_argument("--video-id", default=None,
                    help="id such as clip_001 (default: SHA-256 prefix)")
    ap.add_argument("--force", action="store_true", help="ignore the cache and re-run")
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    set_seed(cfg["seed"])
    try:
        ok, video_id = ingest(cfg, args.video, args.video_id, force=args.force)
    except IngestionError as exc:
        print(f"INGESTION FAILED: {exc}", file=sys.stderr)
        return 2
    print(f"video_id={video_id}  phase1_gate={'PASS' if ok else 'FAIL'}")
    return 0 if ok else 1


if __name__ == "__main__":
    sys.exit(main())
```

11. env_check.py

Phase 0 environment report (imports, versions, CUDA, ffmpeg).

```python
"""Phase 0 environment check: imports, versions, CUDA, ffmpeg -> results/environment_report.json."""
from __future__ import annotations

import importlib
import json
import platform
import shutil
import subprocess
import sys
from datetime import datetime
from pathlib import Path

REQUIRED = ["yaml", "numpy", "cv2", "pytest"]                # needed from Phase 1
OPTIONAL = ["torch", "whisper", "yt_dlp", "scenedetect", "insightface", "open_clip"]


def version_of(name: str):
    try:
        mod = importlib.import_module(name)
        return getattr(mod, "__version__", "ok")
    except Exception as exc:                                 # noqa: BLE001
        return f"MISSING ({type(exc).__name__})"


def tool_version(tool: str):
    if shutil.which(tool) is None:
        return None
    out = subprocess.run([tool, "-version"], capture_output=True, text=True)
    return out.stdout.splitlines()[0] if out.returncode == 0 and out.stdout else None


def main() -> int:
    report = {"python": platform.python_version(), "platform": platform.platform()}
    failed = []
    for name in REQUIRED:
        report[name] = version_of(name)
        if str(report[name]).startswith("MISSING"):
            failed.append(name)
    for name in OPTIONAL:
        report[name] = version_of(name)
    for tool in ("ffmpeg", "ffprobe"):
        report[tool] = tool_version(tool)
        if report[tool] is None:
            failed.append(tool)
    try:
        import torch
        report["cuda"] = bool(torch.cuda.is_available())
    except Exception:                                        # noqa: BLE001
        report["cuda"] = False
    report["timestamp"] = datetime.now().isoformat(timespec="seconds")

    out = Path("results") / "environment_report.json"
    out.parent.mkdir(exist_ok=True)
    out.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps(report, indent=2))
    if failed:
        print("FAILED:", ", ".join(failed), file=sys.stderr)
        return 1
    print("Environment OK")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

12. tests/conftest.py

Fixtures that generate tiny synthetic videos with FFmpeg.

```python
"""Shared fixtures: tiny synthetic videos generated with FFmpeg (no sample files needed)."""
from __future__ import annotations

import shutil
import subprocess
from pathlib import Path

import pytest

from src.common.utils import close_logger, load_config

ROOT = Path(__file__).resolve().parents[1]
needs_ffmpeg = pytest.mark.skipif(shutil.which("ffmpeg") is None, reason="ffmpeg not installed")


def make_video(path: Path, duration: float = 2.0, size: str = "320x240", fps: int = 25,
               audio: bool = True, in_opts=None, vf: str | None = None, extra=None) -> Path:
    cmd = ["ffmpeg", "-y", "-v", "error"] + list(in_opts or [])
    cmd += ["-f", "lavfi", "-i", f"testsrc=duration={duration}:size={size}:rate={fps}"]
    if audio:
        cmd += ["-f", "lavfi", "-i", f"sine=frequency=440:duration={duration}"]
    if vf:
        cmd += ["-vf", vf]
    cmd += ["-c:v", "libx264", "-pix_fmt", "yuv420p"]
    cmd += ["-c:a", "aac"] if audio else ["-an"]
    cmd += list(extra or []) + [str(path)]
    subprocess.run(cmd, check=True, capture_output=True)
    return path


@pytest.fixture
def cfg(tmp_path):
    c = load_config(ROOT / "configs" / "phase1.yaml")
    c["data_dir"] = str(tmp_path / "data")
    c["results_dir"] = str(tmp_path / "results")
    c["log_dir"] = str(tmp_path / "logs")
    c["ingestion"]["random_frame_checks"] = 10
    return c


@pytest.fixture
def make(tmp_path):
    def _make(name="in.mp4", **kw):
        return make_video(tmp_path / name, **kw)
    return _make


@pytest.fixture
def run_ingest(cfg):
    from src.ingestion.video_ingestion import ingest

    def _run(source, video_id="t001", force=False):
        result = ingest(cfg, str(source), video_id, force=force)
        import logging
        close_logger(logging.getLogger(f"cinecomic.{video_id}"))
        return result
    return _run
```

13. tests/test_phase1.py

15 unit tests covering the guide's Phase 1 tests and more.

```python
"""Phase 1 unit tests (plus the Phase 0 config / ffmpeg checks it depends on)."""
from __future__ import annotations

import json
import subprocess
from pathlib import Path

import pytest

from src.common.utils import ConfigError, load_config
from src.ingestion import metadata as md
from src.ingestion import video_loader as vl
from tests.conftest import ROOT, needs_ffmpeg

pytestmark = needs_ffmpeg


def _meta(cfg, vid="t001"):
    return json.loads((Path(cfg["data_dir"]) / "metadata" / f"{vid}.json").read_text())


# ---- Phase 0 checks used by Phase 1 ------------------------------------------------
def test_ffmpeg_callable():
    assert subprocess.run(["ffmpeg", "-version"], capture_output=True).returncode == 0


def test_config_loader_rejects_unknown_keys(tmp_path):
    bad = tmp_path / "bad.yaml"
    bad.write_text("seed: 1\nnot_a_key: 5\n")
    with pytest.raises(ConfigError):
        load_config(bad)
    nested = tmp_path / "bad2.yaml"
    nested.write_text("ingestion:\n  typo_key: 1\n")
    with pytest.raises(ConfigError):
        load_config(nested)


def test_default_config_loads():
    cfg = load_config(ROOT / "configs" / "phase1.yaml")
    assert cfg["ingestion"]["audio_sample_rate"] == 16000


# ---- Phase 1 behaviour -------------------------------------------------------------
def test_happy_path_passes_gate(cfg, make, run_ingest):
    ok, vid = run_ingest(make())
    assert ok and vid == "t001"
    meta = _meta(cfg)
    assert meta["has_audio"] and meta["width"] == 320 and meta["height"] == 240
    assert abs(meta["fps"] - 25) < 0.01 and meta["n_frames"] == 50
    wav = Path(meta["audio_path"])
    assert md.wav_format(wav) == (16000, 1)                    # 16 kHz mono
    assert abs(md.wav_duration(wav) - meta["duration_s"]) <= cfg["gates"]["audio_video_diff_s"]
    metrics = json.loads((Path(cfg["data_dir"]) / "metadata" / "t001.metrics.json").read_text())
    assert metrics["passed"] is True and metrics["decode_errors"] == 0


def test_default_video_id_is_sha256_prefix(cfg, make, run_ingest, tmp_path):
    src = make()
    ok, vid = run_ingest(src, video_id=None)
    assert ok and vid == vl.sha256_file(src)[:12]


def test_truncated_file_rejected(cfg, make, run_ingest, tmp_path):
    good = make("good.mp4")
    data = good.read_bytes()
    broken = tmp_path / "broken.mp4"
    broken.write_bytes(data[: len(data) // 2])                 # moov atom is lost
    with pytest.raises(md.IngestionError):
        run_ingest(broken)


def test_missing_and_empty_files_rejected(run_ingest, tmp_path):
    with pytest.raises(md.IngestionError):
        run_ingest(tmp_path / "nope.mp4")
    empty = tmp_path / "empty.mp4"
    empty.write_bytes(b"")
    with pytest.raises(md.IngestionError):
        run_ingest(empty)


def test_no_audio_sets_flag_and_skips_wav(cfg, make, run_ingest):
    ok, _ = run_ingest(make(audio=False))
```

```
    meta = _meta(cfg)
    assert ok and meta["has_audio"] is False and meta["audio_path"] is None
    assert not (Path(cfg["data_dir"]) / "audio" / "t001.wav").exists()


def test_rotated_video_is_upright(cfg, make, run_ingest):
    # Build a rotated file by remuxing a normal one with a display-rotation tag.
    base = make("base.mp4", size="320x240")
    rotated = base.parent / "rotated.mp4"
    subprocess.run(["ffmpeg", "-y", "-v", "error", "-display_rotation", "90", "-i", str(base),
                    "-c", "copy", str(rotated)], check=True, capture_output=True)
    assert md.describe(rotated)["rotation"] in (90, 270)
    assert md.describe(rotated)["height"] == 320               # displayed portrait
    ok, _ = run_ingest(rotated)
    meta = _meta(cfg)
    assert ok and meta["normalised"] and meta["source_rotation"] in (90, 270)
    final = md.describe(Path(meta["raw_path"]))
    assert final["rotation"] == 0 and final["width"] == 240 and final["height"] == 320


def test_variable_frame_rate_becomes_cfr(cfg, make, run_ingest):
    vfr = make("vfr.mp4", duration=4, fps=30, vf="select='not(mod(n,3))+not(mod(n,7))'",
               extra=["-fps_mode", "vfr"])
    if not md.describe(vfr)["vfr"]:
        pytest.skip("could not synthesise a VFR file with this FFmpeg build")
    ok, _ = run_ingest(vfr)
    meta = _meta(cfg)
    assert ok and meta["source_vfr"] is True and meta["vfr"] is False and meta["normalised"]


def test_max_duration_is_enforced(cfg, make, run_ingest):
    cfg["ingestion"]["max_duration_s"] = 1
    ok, _ = run_ingest(make(duration=3))
    assert ok and _meta(cfg)["duration_s"] <= 1.1


def test_too_short_video_rejected(cfg, make, run_ingest):
    cfg["ingestion"]["min_duration_s"] = 5
    with pytest.raises(md.IngestionError):
        run_ingest(make(duration=2))


def test_cache_skips_second_run_and_force_reruns(cfg, make, run_ingest):
    src = make()
    run_ingest(src)
    mp = Path(cfg["data_dir"]) / "metadata" / "t001.metrics.json"
    first = mp.stat().st_mtime_ns
    run_ingest(src)                                            # cached
    assert mp.stat().st_mtime_ns == first
    run_ingest(src, force=True)                                # forced
    assert mp.stat().st_mtime_ns > first


def test_same_input_same_metadata(cfg, make, run_ingest):
    src = make()
    run_ingest(src, "a1")
    run_ingest(src, "a2")
    a, b = _meta(cfg, "a1"), _meta(cfg, "a2")
    for key in ("duration_s", "fps", "width", "height", "n_frames", "audio_duration_s"):
        assert a[key] == b[key]


def test_gate_flags_black_video(cfg, make, run_ingest, tmp_path):
    black = tmp_path / "black.mp4"
    subprocess.run(["ffmpeg", "-y", "-v", "error", "-f", "lavfi", "-i",
                    "color=c=black:s=320x240:d=2:r=25", "-f", "lavfi", "-i",
                    "sine=frequency=440:duration=2", "-c:v", "libx264", "-pix_fmt", "yuv420p",
                    "-c:a", "aac", str(black)], check=True, capture_output=True)
    ok, _ = run_ingest(black)
    assert ok is False                                         # random-frame check fails
```

Test results

Run with `python -m pytest -q tests`: 15 passed.

Test	What it proves
`test_ffmpeg_callable`	ffmpeg runs (Phase 0 gate)
`test_config_loader_rejects_unknown_keys`	typos in YAML are caught, including nested ones
`test_default_config_loads`	configs/phase1.yaml is valid
`test_happy_path_passes_gate`	metadata correct, WAV is 16 kHz mono, A/V length within 0.1 s, 0 decode errors
`test_default_video_id_is_sha256_prefix`	video_id defaults to SHA-256 prefix
`test_truncated_file_rejected`	truncated file raises a clear IngestionError
`test_missing_and_empty_files_rejected`	missing or empty input rejected
`test_no_audio_sets_flag_and_skips_wav`	has_audio=false, no WAV written
`test_rotated_video_is_upright`	phone-style rotated video becomes upright, rotation tag removed
`test_variable_frame_rate_becomes_cfr`	VFR source re-encoded to CFR
`test_max_duration_is_enforced`	development duration cap works
`test_too_short_video_rejected`	min duration enforced
`test_cache_skips_second_run_and_force_reruns`	caching by config hash; --force re-runs
`test_same_input_same_metadata`	determinism
`test_gate_flags_black_video`	random-frame check makes the gate fail on an all-black video

Next step

Run Phase 1 on your 5-10 labelled clips and confirm every gate passes. Then Phase 2 (Whisper) reads `data/audio/{video_id}.wav` and `data/metadata/{video_id}.json`; it will subclass BaseStage in `src/audio/whisper_asr.py` and write data/subtitles/{video_id}.json.